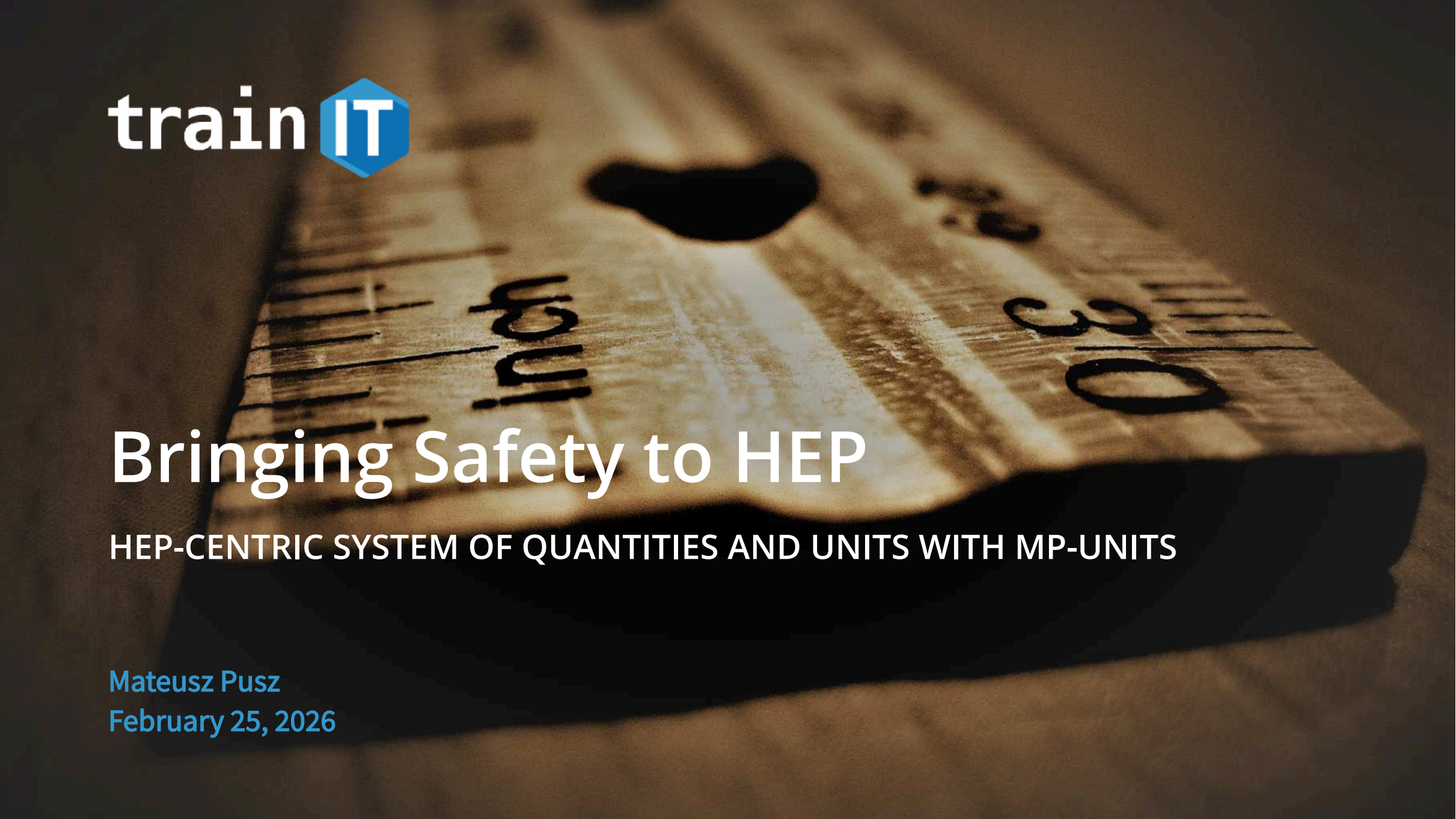


Bringing Safety to HEP

HEP-CENTRIC SYSTEM OF QUANTITIES AND UNITS WITH MP-UNITS

Mateusz Pusz

February 25, 2026

Who am I?

- **Modern C++ Evangelist**
- Hacking C++ for more than *20 years* for fun and living

Who am I?

- **Modern C++ Evangelist**
- Hacking C++ for more than *20 years* for fun and living
- Founder and the main author of the **mp-units** Open Source library
- Active voting member and *contributor* of the **ISO C++ Committee** (WG21)

Who am I?

- **Modern C++ Evangelist**
- Hacking C++ for more than *20 years* for fun and living
- Founder and the main author of the **mp-units** Open Source library
- Active voting member and *contributor* of the **ISO C++ Committee** (WG21)
- *Trainer*, *coach*, *mentor*, *consultant*, and *conference speaker*

Who am I?

- **Modern C++ Evangelist**
- Hacking C++ for more than *20 years* for fun and living
- Founder and the main author of the **mp-units** Open Source library
- Active voting member and *contributor* of the **ISO C++ Committee** (WG21)
- *Trainer*, *coach*, *mentor*, *consultant*, and *conference speaker*

As a freelancer, hours spent on Open Source and ISO C++ Committee work compete directly with billable time. If you find this work valuable, please consider sponsoring or engaging me for training.

Disclaimer

My expertise: C++ type systems and zero-cost abstractions.
Your expertise: HEP physics.
Together: catching bugs before they publish papers! 🎯

Disclaimer

My expertise: C++ type systems and zero-cost abstractions.
Your expertise: HEP physics.
Together: catching bugs before they publish papers! 🎯

LLMs helped me understand HEP domain and generate proper examples. If details need refinement, please share—that's exactly the domain knowledge we want to encode into types!

C++ Safety & Scientific Correctness

In High-Energy Physics, **Safety = Correctness and Reproducibility**.

C++ Safety & Scientific Correctness

In High-Energy Physics, Safety = Correctness and Reproducibility.

- We trust software with
  - Complex *detector geometries* (millions of volumes)
  - Critical *tracking and reconstruction algorithms*
  - Multi-day *simulations* on the Grid

C++ Safety & Scientific Correctness

In High-Energy Physics, Safety = Correctness and Reproducibility.

- We trust software with
  - Complex *detector geometries* (millions of volumes)
  - Critical *tracking and reconstruction algorithms*
  - Multi-day *simulations* on the Grid

The Problem: A single unit mismatch in a **`double`** could silently invalidate a result or cause a 10x scale error in detector simulation.

The Challenge at CERN

- **2.5M lines of code** in ATLAS Athena framework alone
- Multiple frameworks with different unit conventions
  - **Geant4**: mm, ns, MeV, rad
  - **ROOT**: cm, s, GeV, degree
  - **CLHEP**: Different CODATA versions (2014, 2018, 2022)

The Challenge at CERN

- **2.5M lines of code** in ATLAS Athena framework alone
- Multiple frameworks with different unit conventions
  - **Geant4**: mm, ns, MeV, rad
  - **ROOT**: cm, s, GeV, degree
  - **CLHEP**: Different CODATA versions (2014, 2018, 2022)

Training alone won't solve this

- Code often written by physicists, not software engineers
- Silent bugs from **`double`** proliferation
- Compiler can't help when everything is dimensionless

The "Argument Soup" Problem

GEANT4: `G4Trap` CONSTRUCTOR

```
G4Trap(const G4String& name,
       G4double pDz, G4double pTheta, G4double pPhi,
       G4double pDy1, G4double pDx1, G4double pDx2, G4double pAlp1,
       G4double pDy2, G4double pDx3, G4double pDx4, G4double pAlp2);
```

The "Argument Soup" Problem

GEANT4: `G4Trap` CONSTRUCTOR

```
G4Trap(const G4String& name,
       G4double pDz, G4double pTheta, G4double pPhi,
       G4double pDy1, G4double pDx1, G4double pDx2, G4double pAlp1,
       G4double pDy2, G4double pDx3, G4double pDx4, G4double pAlp2);
```

- **12 double parameters** - which are lengths? which are angles?
- Accidentally swapping **`pDx2`** and **`pAlp1`** compiles fine

The "Argument Soup" Problem

GEANT4: `G4Trap` CONSTRUCTOR

```
G4Trap(const G4String& name,
       G4double pDz, G4double pTheta, G4double pPhi,
       G4double pDy1, G4double pDx1, G4double pDx2, G4double pAlp1,
       G4double pDy2, G4double pDx3, G4double pDx4, G4double pAlp2);
```

- **12 double parameters** - which are lengths? which are angles?
- Accidentally swapping **`pDx2`** and **`pAlp1`** compiles fine

Result: Invalid detector geometry, caught only at runtime (maybe)

The "Argument Soup" Problem

ROOT: `TGeoMedium` CONSTRUCTOR

```
TGeoMedium(const char *name, Int_t numed, TGeoMaterial *mat,
           Double_t fieldm, Double_t tmaxfd, Double_t stemax,
           Double_t deemax, Double_t epsil, Double_t stmin);
```

The "Argument Soup" Problem

ROOT: `TGeoMedium` CONSTRUCTOR

```
TGeoMedium(const char *name, Int_t numed, TGeoMaterial *mat,
           Double_t fieldm, Double_t tmaxfd, Double_t stemax,
           Double_t deemax, Double_t epsil, Double_t stmin);
```

- **6 `Double_t` parameters after the material**
- **`fieldm`**: magnetic field (G), **`tmaxfd`**: angle (deg), **`stemax`**: length (cm)
- **`deemax`**: energy ratio (dimensionless), **`epsil`**: (dimensionless), **`stmin`**: length (cm)

The "Argument Soup" Problem

ROOT: `TGeoMedium` CONSTRUCTOR

```
TGeoMedium(const char *name, Int_t numed, TGeoMaterial *mat,
           Double_t fieldm, Double_t tmaxfd, Double_t stemax,
           Double_t deemax, Double_t epsil, Double_t stmin);
```

- **6 `Double_t` parameters after the material**
- **`fieldm`**: magnetic field (G), **`tmaxfd`**: angle (deg), **`stemax`**: length (cm)
- **`deemax`**: energy ratio (dimensionless), **`epsil`**: (dimensionless), **`stmin`**: length (cm)

All silently convertible to each other!

The "Argument Soup" Problem

GEANT4 PARTICLE DEFINITION

```
// G4ParticleDefinition constructor: 21 parameters!
G4ParticleDefinition(const G4String& aName,
                     G4double mass, G4double width, G4double charge,
                     G4int iSpin, G4int iParity, G4int iConjugation,
                     G4int iIsospin, G4int iIsospinZ, G4int gParity,
                     const G4String& pType, G4int lepton, G4int baryon,
                     G4int encoding, G4bool stable, G4double lifetime,
                     G4DecayTable* decaytable, G4bool shortlived,
                     const G4String& subType, G4int anti_encoding,
                     G4double magneticMoment);
```

The "Argument Soup" Problem

GEANT4 PARTICLE DEFINITION

```
// G4ParticleDefinition constructor: 21 parameters!
G4ParticleDefinition(const G4String& aName,
                     G4double mass, G4double width, G4double charge,
                     G4int iSpin, G4int iParity, G4int iConjugation,
                     G4int iIsospin, G4int iIsospinZ, G4int gParity,
                     const G4String& pType, G4int lepton, G4int baryon,
                     G4int encoding, G4bool stable, G4double lifetime,
                     G4DecayTable* decaytable, G4bool shortlived,
                     const G4String& subType, G4int anti_encoding,
                     G4double magneticMoment);
```

Multiple dangerous pairs of same-typed parameters:

- **`mass`**, **`width`**, **`lifetime`**, **`magneticMoment`** - all **`G4double`** (energy/time)
- **`iSpin`**, **`iParity`**, **`iConjugation`**, **`iIsospin`**, **`iIsospinZ`**, **`gParity`** - all **`G4int`**

Potential HEP Bug: Parameter Swapping

```
using namespace CLHEP;

// MeV, GeV are just 'constexpr double' scale factors!
double mass = 3.872 * GeV;  // rest energy
double width = 1.2 * MeV;   // decay width

G4ParticleDefinition("X3872", width, mass, 0, ...);
```

Potential HEP Bug: Parameter Swapping

```
using namespace CLHEP;

// MeV, GeV are just 'constexpr double' scale factors!
double mass = 3.872 * GeV;  // rest energy
double width = 1.2 * MeV;   // decay width

G4ParticleDefinition("X3872", width, mass, 0, ...);
```

- *Compiles and runs without error* (both are **G4double**)
- Particle rest energy = 1.2 MeV (should be 3872 MeV) - factor of **3000× wrong**!
- Decay width = 3872 MeV (should be 1.2 MeV) - **absurdly large**
- Wrong decay kinematics, invalid cross-sections
- **Caught only in physics validation (or never!)**

The "Magic 1.0" Problem

Different frameworks use different internal base units

QUANTITY	GEANT4	ROOT
Length	`1.0` = 1 mm	`1.0` = 1 cm
Time	`1.0` = 1 ns	`1.0` = 1 s
Angle	`1.0` = 1 rad	`1.0` = 1 degree
Energy	`1.0` = 1 MeV	`1.0` = 1 GeV

The "Magic 1.0" Problem

```
// Legacy Bridge Code
double radius = rootVolume->GetRmax(); // Returns 10.0 (meaning 10 cm)

// Geant4 expects mm internally
G4Tubs g4Tube("Tube", 0, radius, ...);
// Interprets as 10 mm!
```

The "Magic 1.0" Problem

```
// Legacy Bridge Code
double radius = rootVolume->GetRmax(); // Returns 10.0 (meaning 10 cm)

// Geant4 expects mm internally
G4Tubs g4Tube("Tube", 0, radius, ...);
// Interprets as 10 mm!
```

Result: Your detector geometry is now **10× smaller** silently.

The "Magic 1.0" Problem

```
// Legacy Bridge Code
double radius = rootVolume->GetRmax(); // Returns 10.0 (meaning 10 cm)

// Geant4 expects mm internally
G4Tubs g4Tube("Tube", 0, radius, ...);
// Interprets as 10 mm!
```

Result: Your detector geometry is now **10× smaller** silently.

Examples from real ATLAS/CMS code

- **TGeoCone** (ROOT) vs **G4Cons** (Geant4): parameter order AND units differ
- Bridging code requires manual scaling factors
- One missing **/CLHEP::cm** → 10× error

The CODATA Drift Problem

Different CERN frameworks use different physics constants:

FRAMEWORK	CODATA VERSION	ELECTRON MASS
CLHEP (old)	2014	0.5109989461 MeV
Gaudi (old)	2014	0.5109989461 MeV
Geant4 (current)	2018	0.51099895000 MeV
ROOT (latest)	2022	0.51099895069 MeV

The CODATA Drift Problem

Different CERN frameworks use different physics constants:

FRAMEWORK	CODATA VERSION	ELECTRON MASS
CLHEP (old)	2014	0.5109989461 MeV
Gaudi (old)	2014	0.5109989461 MeV
Geant4 (current)	2018	0.51099895000 MeV
ROOT (latest)	2022	0.51099895069 MeV

- Small differences, but **accumulate** in precision calculations
- Mixing frameworks → inconsistent physics
- **Current solution**: hope everyone uses same version
- **mp-units solution**: Compile-time namespace selection

mp-units: Design Philosophy

COMPILE-TIME SAFETY

- Correct handling of physical quantities, units, and numerical values
- Zero runtime overhead

mp-units: Design Philosophy

COMPILE-TIME SAFETY

- Correct handling of physical quantities, units, and numerical values
- Zero runtime overhead

PERFORMANCE

- As fast as working with fundamental types
- Same assembly as hand-written **`double`** code

mp-units: Design Philosophy

COMPILE-TIME SAFETY

- Correct handling of physical quantities, units, and numerical values
- Zero runtime overhead

PERFORMANCE

- As fast as working with fundamental types
- Same assembly as hand-written **`double`** code

GREAT USER EXPERIENCE

- Modern C++ (NTTPs, concepts, formatting)
- Optimized for readable compilation errors

mp-units: Design Philosophy

COMPILE-TIME SAFETY

- Correct handling of physical quantities, units, and numerical values
- Zero runtime overhead

PERFORMANCE

- As fast as working with fundamental types
- Same assembly as hand-written **`double`** code

GREAT USER EXPERIENCE

- Modern C++ (NTTPs, concepts, formatting)
- Optimized for readable compilation errors

HEP-SPECIFIC FEATURES

- Dedicated system of quantities
- CODATA version selection (2014, 2018, 2022)
- Scalar, vector, and tensor quantities
- Easy to extend for custom experiment units

mp-units: Design Philosophy

COMPILE-TIME SAFETY

- Correct handling of physical quantities, units, and numerical values
- Zero runtime overhead

PERFORMANCE

- As fast as working with fundamental types
- Same assembly as hand-written **`double`** code

GREAT USER EXPERIENCE

- Modern C++ (NTTPs, concepts, formatting)
- Optimized for readable compilation errors

HEP-SPECIFIC FEATURES

- Dedicated system of quantities
- CODATA version selection (2014, 2018, 2022)
- Scalar, vector, and tensor quantities
- Easy to extend for custom experiment units

STANDARDIZATION

- Foundation for future **`std::units`**
- ISO C++ proposals P1935, P2980, P3045

Six Levels of Safety

1. **Dimension Safety** - Prevents mixing incompatible dimensions

Six Levels of Safety

1. **Dimension Safety** - Prevents mixing incompatible dimensions
2. **Unit Safety** - Prevents unit mismatches and eliminates manual scaling factors

Six Levels of Safety

1. **Dimension Safety** - Prevents mixing incompatible dimensions
2. **Unit Safety** - Prevents unit mismatches and eliminates manual scaling factors
3. **Representation Safety** - Protects against overflows and precision loss

Six Levels of Safety

1. **Dimension Safety** - Prevents mixing incompatible dimensions
2. **Unit Safety** - Prevents unit mismatches and eliminates manual scaling factors
3. **Representation Safety** - Protects against overflows and precision loss
4. **Quantity Kind Safety** - Prevents arithmetic on quantities of different kinds

Six Levels of Safety

1. **Dimension Safety** - Prevents mixing incompatible dimensions
2. **Unit Safety** - Prevents unit mismatches and eliminates manual scaling factors
3. **Representation Safety** - Protects against overflows and precision loss
4. **Quantity Kind Safety** - Prevents arithmetic on quantities of different kinds
5. **Quantity Safety** - Enforces correct quantity relationships and equations

Six Levels of Safety

1. **Dimension Safety** - Prevents mixing incompatible dimensions
2. **Unit Safety** - Prevents unit mismatches and eliminates manual scaling factors
3. **Representation Safety** - Protects against overflows and precision loss
4. **Quantity Kind Safety** - Prevents arithmetic on quantities of different kinds
5. **Quantity Safety** - Enforces correct quantity relationships and equations
6. **Affine Space Safety** - Distinguishes points and deltas

Six Levels of Safety

1. **Dimension Safety** - Prevents mixing incompatible dimensions
2. **Unit Safety** - Prevents unit mismatches and eliminates manual scaling factors
3. **Representation Safety** - Protects against overflows and precision loss
4. **Quantity Kind Safety** - Prevents arithmetic on quantities of different kinds
5. **Quantity Safety** - Enforces correct quantity relationships and equations
6. **Affine Space Safety** - Distinguishes points and deltas

Each level adds another layer of protection, encoding domain knowledge directly into the type system.

Level 1: Dimension Safety

The real danger: wrong arguments in function calls

```
using namespace mp_units::hep::unit_symbols;

// Function expecting distance first, then time
quantity<mm / ns> average_speed(quantity<mm> distance, quantity<ns> time);
```

```
quantity bunch_time   = 25 * ns;   // LHC bunch spacing
quantity track_length = 120 * mm;  // Silicon tracker path

quantity speed = average_speed(track_length, bunch_time);  // ✅ Correct

// auto bad1 = average_speed(bunch_time, track_length);    // ❌ Compile error: args swapped!
// auto bad2 = average_speed(track_length, 45 * MeV);      // ❌ Compile error: energy ≠ time!
```

Level 2: Unit Safety

Automatic, zero-cost unit conversions

```
using namespace mp_units::hep::unit_symbols;

quantity g4_radius = 150. * mm;            // Geant4 uses mm
quantity root_radius = g4_radius.in(cm);   // ROOT uses cm

// Automatic conversion, zero runtime cost
assert(g4_radius == root_radius);          // ✅ Compiles and passes!

// Extract value in specific unit for legacy APIs
double g4_val = g4_radius.numerical_value_in(mm);     // 150.0
double root_val = root_radius.numerical_value_in(cm); // 15.0
```

Level 2: Unit Safety

Automatic, zero-cost unit conversions

```
using namespace mp_units::hep::unit_symbols;

quantity g4_radius = 150. * mm;            // Geant4 uses mm
quantity root_radius = g4_radius.in(cm);   // ROOT uses cm

// Automatic conversion, zero runtime cost
assert(g4_radius == root_radius);          // ✅ Compiles and passes!

// Extract value in specific unit for legacy APIs
double g4_val = g4_radius.numerical_value_in(mm);     // 150.0
double root_val = root_radius.numerical_value_in(cm); // 15.0
```

Eliminates the ROOT/Geant4 bridge bugs:

- No manual `* 10.0` scaling factors
- Compiler handles conversions
- If units match, conversion optimizes away completely

Level 3: Representation Safety

Preventing truncation and precision loss

```
using namespace mp_units::hep::unit_symbols;

// Raw ADC readout from calorimeter
quantity<one, std::uint16_t> raw_adc = 3500;                  // 12-bit ADC

// Calibration: ADC → voltage → energy
constexpr quantity adc_to_voltage = 2500.0 * mV / 4096;  // ADC → voltage
constexpr quantity energy_scale = 45.0 * MeV / V;        // Voltage → energy
quantity voltage = raw_adc * adc_to_voltage;             // Auto-upgrades to double
quantity energy = voltage * energy_scale;                // ~96 MeV

// ❌ ERROR: Narrowing conversion
// quantity<mV, std::uint16_t> v_int = voltage;          // Compile-time error

// ✅ CORRECT: Explicit truncation when needed
quantity<mV, std::uint16_t> v_int = voltage.force_in<std::uint16_t>(mV); // Fractional mV lost!
```

Level 4: Quantity Kind Safety

*Incompatible quantities with the **same dimension** (T) — but different (sub)kind*

```
using namespace mp_units::hep::unit_symbols;

quantity<one> calc_gamma(quantity<hep::proper_time[ns]>     tau,
                         quantity<hep::coordinate_time[ns]> t_lab);
```

```
quantity tau   = hep::proper_time(0.385 * ns);     // B⁰ meson rest-frame lifetime
quantity t_lab = hep::coordinate_time(4.2 * ns);   // observed lab time of flight

quantity gamma = calc_gamma(tau, t_lab);           // ✅ Correct
// calc_gamma(t_lab, tau);                         // ❌ args swapped — kind mismatch!
// auto sum = tau + t_lab;                         // ❌ can't add different kinds!
// bool cmp = tau < t_lab;                         // ❌ can't compare different kinds!
```

Level 4: Quantity Kind Safety

*Incompatible quantities with the **same dimension** (T) — but different (sub)kind*

```
using namespace mp_units::hep::unit_symbols;

quantity<one> calc_gamma(quantity<hep::proper_time[ns]>     tau,
                         quantity<hep::coordinate_time[ns]> t_lab);
```

```
quantity tau   = hep::proper_time(0.385 * ns);     // B⁰ meson rest-frame lifetime
quantity t_lab = hep::coordinate_time(4.2 * ns);   // observed lab time of flight

quantity gamma = calc_gamma(tau, t_lab);           // ✅ Correct
// calc_gamma(t_lab, tau);                         // ❌ args swapped — kind mismatch!
// auto sum = tau + t_lab;                         // ❌ can't add different kinds!
// bool cmp = tau < t_lab;                         // ❌ can't compare different kinds!
```

- Hz vs Bq, Sv vs Gy, rad vs sr (if dimensionless)

Level 5: Quantity Safety

*Turning the 11-**`double`** soup into a self-documenting API*

```
// Each geometric axis and angle role gets its own sub-kind of hep::length / hep::angle
inline constexpr struct trap_half_z      final : quantity_spec<hep::length> {} trap_half_z;
inline constexpr struct trap_half_y      final : quantity_spec<hep::length> {} trap_half_y;
inline constexpr struct trap_half_x      final : quantity_spec<hep::length> {} trap_half_x;
inline constexpr struct trap_tilt_angle  final : quantity_spec<hep::angle>  {} trap_tilt_angle;
inline constexpr struct trap_shear_angle final : quantity_spec<hep::angle>  {} trap_shear_angle;
```

```
void MakeG4Trap(quantity<trap_half_z[mm]>       pDz,
                quantity<trap_tilt_angle[rad]>  pTheta,
                quantity<trap_tilt_angle[rad]>  pPhi,
                quantity<trap_half_y[mm]>       pDy1,
                quantity<trap_half_x[mm]>       pDx1,
                quantity<trap_half_x[mm]>       pDx2,
                quantity<trap_shear_angle[rad]> pAlp1,
                quantity<trap_half_y[mm]>       pDy2,
                quantity<trap_half_x[mm]>       pDx3,
                quantity<trap_half_x[mm]>       pDx4,
                quantity<trap_shear_angle[rad]> pAlp2);
```

```
quantity pDz    = trap_half_z(150. * mm);
quantity pDy1   = trap_half_y(25. * mm);
quantity theta = trap_tilt_angle(0. * rad);

// ❌ Level 1: mm where rad expected (pDy1)!
// MakeG4Trap(pDz, pDy1, ...);
// ❌ Level 5: pDy passed where pDz expected (both mm)!
// MakeG4Trap(pDy1, theta, ...);

// ✅ Lengths of different axes promote to
// common hep::length — safe for clearance calcs
quantity half_diag = pDz + pDy1;
static_assert(half_diag.quantity_spec == hep::length);
```

Level 6: Affine Space Safety

Distinguishing points and deltas

```
using namespace mp_units::hep::unit_symbols;

// Points: absolute positions in detector
quantity_point z_vertex = z_vertex_zero + 15.0 * mm;
quantity_point z_decay = z_vertex_zero + 20.2 * mm;

// Point - Point = Delta (displacement) ✅
quantity decay_length = z_decay - z_vertex;  // 5.2 mm

// Point + Delta = Point (new position) ✅
quantity_point z_final = z_vertex + decay_length;

// ❌ ERROR: Cannot add absolute positions
// auto bad = z_vertex + z_decay;  // Compile error!
```

Bonus: Generic Interfaces

Unit-agnostic algorithms that preserve input units

```
using namespace mp_units::hep::unit_symbols;

// Algorithm is unit-agnostic: accepts mm, cm, m, etc. for position coordinates
QuantityPointOf<hep::position_vector> auto decay_vertex_position(QuantityPointOf<hep::position_vector> auto production_z,
                                                                 QuantityOf<hep::decay_length> auto decay_length,
                                                                 QuantityOf<hep::angle> auto theta)
{
  return production_z + decay_length * hep::cos(theta);  // Returns position in same unit
}

// Call with mm (Geant4) → returns position in mm
quantity_point z_decay_g4 = decay_vertex_position(point<mm>(150), 25 * mm, 18 * hep::degree);

// Call with cm (ROOT) → returns position in cm
quantity_point z_decay_root = decay_vertex_position(point<cm>(15), 2.5 * cm, 0.314 * hep::radian);
```

Benefits: Zero conversion overhead, natural units preserved, type-safe

Bonus: Strongly-Typed Dimensionless Quantities

Preventing confusion between incompatible dimensionless values

```
inline constexpr struct efficiency final : quantity_spec<dimensionless, is_kind> {} efficiency;
inline constexpr struct branching_ratio final : quantity_spec<dimensionless, is_kind> {} branching_ratio;
inline constexpr struct event_count final : quantity_spec<dimensionless, is_kind> {} event_count;
inline constexpr struct yield final : quantity_spec<dimensionless, is_kind> {} yield;

quantity<yield[one]> compute_yield(quantity<efficiency[one]> eff,
                                   quantity<event_count[one], int> n_events);

quantity trigger_eff = efficiency(0.95);
quantity selection_eff = efficiency(0.87);
quantity BR_Hgg = branching_ratio(2.27e-3);

// ❌ ERROR: Cannot add or compare incompatible dimensionless kinds
// auto bad = trigger_eff + BR_Hgg;     // Compile error!
// bool bad2 = trigger_eff > BR_Hgg;    // Compile error!

// ✅ CORRECT: Function enforces kind at call site
quantity res = compute_yield(trigger_eff, event_count(1'000'000));  // ✅ OK
// quantity res = compute_yield(BR_Hgg, event_count(1'000'000));    // ❌ Compile error!
```

Also useful for: purity, asymmetries, χ^2/ndf, ratios, confidence levels

Bonus: Faster-than-Lightspeed Constants

`c` as a compile-time unit — never multiplied at runtime

```
using namespace mp_units::hep::unit_symbols;

// c is a named_constant unit in mp-units, not a floating-point value
quantity p = 4. * GeV / c;      // type: quantity<GeV/c>   — c stays symbolic
quantity mass = 3. * GeV / c2;  // type: quantity<GeV/c²>  — c² stays symbolic
```

Bonus: Faster-than-Lightspeed Constants

`c` as a compile-time unit — never multiplied at runtime

```
using namespace mp_units::hep::unit_symbols;

// c is a named_constant unit in mp-units, not a floating-point value
quantity p = 4. * GeV / c;      // type: quantity<GeV/c>   — c stays symbolic
quantity mass = 3. * GeV / c2;  // type: quantity<GeV/c²>  — c² stays symbolic
```

```
quantity E = hypot(p * c, mass * c2);   // E² = (pc)² + (mc²)²  ⟶  4² + 3² = 25  ⟶  E = 5 GeV
```

Bonus: Faster-than-Lightspeed Constants

`c` as a compile-time unit — never multiplied at runtime

```
using namespace mp_units::hep::unit_symbols;

// c is a named_constant unit in mp-units, not a floating-point value
quantity p = 4. * GeV / c;      // type: quantity<GeV/c>   — c stays symbolic
quantity mass = 3. * GeV / c2;  // type: quantity<GeV/c²>  — c² stays symbolic
```

```
quantity E = hypot(p * c, mass * c2);   // E² = (pc)² + (mc²)²  ⟶  4² + 3² = 25  ⟶  E = 5 GeV
```

```
std::println("p = {} = {::N[.5g]}", p, p.in(kg * m / s));  // "p = 4 GeV/c = 2.1377e-18 m kg/s"
std::println("m = {} = {::N[.5g]}", mass, mass.in(kg));    // "m = 3 GeV/c² = 5.348e-27 kg"
std::println("E = {} = {::N[.5g]}", E, E.in(hep::joule));  // "E = 5 GeV = 8.0109e-10 J"
```

Bonus: Faster-than-Lightspeed Constants

`c` as a compile-time unit — never multiplied at runtime

```
using namespace mp_units::hep::unit_symbols;

// c is a named_constant unit in mp-units, not a floating-point value
quantity p = 4. * GeV / c;      // type: quantity<GeV/c>   — c stays symbolic
quantity mass = 3. * GeV / c2;  // type: quantity<GeV/c²>  — c² stays symbolic
```

```
quantity E = hypot(p * c, mass * c2);   // E² = (pc)² + (mc²)²  ⟶  4² + 3² = 25  ⟶  E = 5 GeV
```

```
std::println("p = {} = {::N[.5g]}", p, p.in(kg * m / s));  // "p = 4 GeV/c = 2.1377e-18 m kg/s"
std::println("m = {} = {::N[.5g]}", mass, mass.in(kg));    // "m = 3 GeV/c² = 5.348e-27 kg"
std::println("E = {} = {::N[.5g]}", E, E.in(hep::joule));  // "E = 5 GeV = 8.0109e-10 J"
```

The **c** factors (and all other constants like **π**) cancel in the type system — identical to working in natural units, with full dimensional safety.

The HEP System: Energy as Base Quantity

Why HEP physics differs from ISQ?

The HEP System: Energy as Base Quantity

Why HEP physics differs from ISQ?

ISQ (INTERNATIONAL SYSTEM OF QUANTITIES)

- *7 base dimensions:* length, mass, time, current, temperature, amount, luminosity
- *Derived:* energy = mass × length^2 / time^2 $(M \cdot L^2 \cdot T^{-2})$

The HEP System: Energy as Base Quantity

Why HEP physics differs from ISQ?

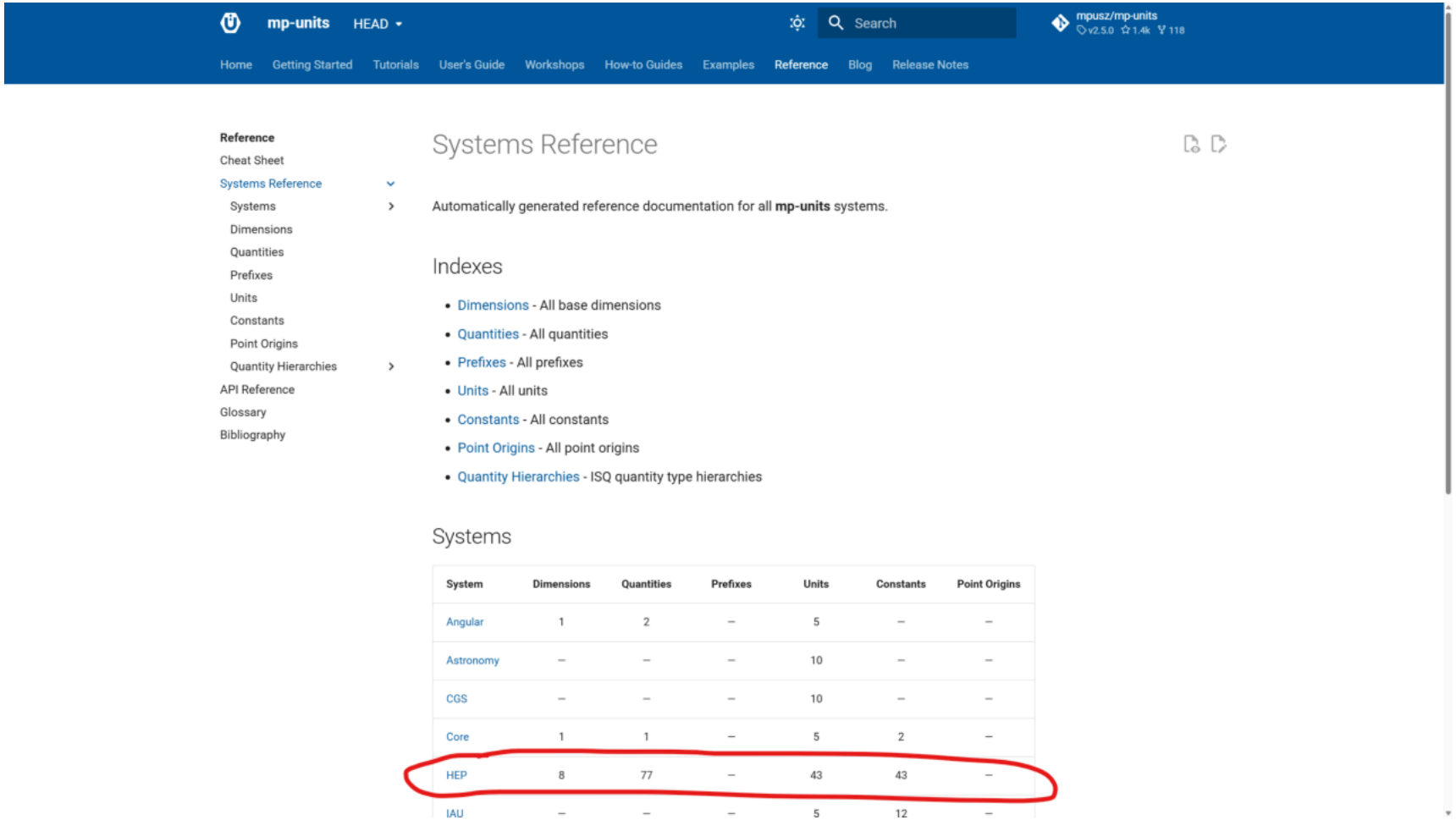
ISQ (INTERNATIONAL SYSTEM OF QUANTITIES)

- *7 base dimensions:* length, mass, time, current, temperature, amount, luminosity
- *Derived:* energy = mass × length^2 / time^2 ($M \cdot L^2 \cdot T^{-2}$)

HEP SYSTEM (HIGH ENERGY PHYSICS)

- **Energy (E)** is a base quantity, not mass!
- **Electric charge (Q)** is base, not current!
- Natural for particle physics: particles characterized by energy, not mass

The HEP System Reference



HEP dimensions and base quantities

```
namespace mp_units::hep {
using namespace ::mp_units::angular;

// dimensions
inline constexpr struct dim_length final : base_dimension<"L"> {} dim_length;
inline constexpr struct dim_time final : base_dimension<"T"> {} dim_time;
inline constexpr struct dim_electric_charge final : base_dimension<"Q"> {} dim_electric_charge;
inline constexpr struct dim_energy final : base_dimension<"E"> {} dim_energy;
inline constexpr struct dim_temperature final : base_dimension<symbol_text{u8"Θ", "O"}> {} dim_temperature;
inline constexpr struct dim_amount_of_substance final : base_dimension<"N"> {} dim_amount_of_substance;
inline constexpr struct dim_luminous_intensity final : base_dimension<"I"> {} dim_luminous_intensity;

// base quantities
inline constexpr struct length final : quantity_spec<dim_length> {} length;
inline constexpr struct duration final : quantity_spec<dim_time> {} duration;
inline constexpr struct electric_charge final : quantity_spec<dim_electric_charge> {} electric_charge;
inline constexpr struct energy final : quantity_spec<dim_energy> {} energy;
inline constexpr struct temperature final : quantity_spec<dim_temperature> {} temperature;
inline constexpr struct amount_of_substance final : quantity_spec<dim_amount_of_substance> {} amount_of_substance;
inline constexpr struct luminous_intensity final : quantity_spec<dim_luminous_intensity> {} luminous_intensity;
}
```

HEP dimensions and base quantities

```
namespace mp_units::hep {
using namespace ::mp_units::angular;

// dimensions
inline constexpr struct dim_length final : base_dimension<"L"> {} dim_length;
inline constexpr struct dim_time final : base_dimension<"T"> {} dim_time;
inline constexpr struct dim_electric_charge final : base_dimension<"Q"> {} dim_electric_charge;
inline constexpr struct dim_energy final : base_dimension<"E"> {} dim_energy;
inline constexpr struct dim_temperature final : base_dimension<symbol_text{u8"Θ", "O"}> {} dim_temperature;
inline constexpr struct dim_amount_of_substance final : base_dimension<"N"> {} dim_amount_of_substance;
inline constexpr struct dim_luminous_intensity final : base_dimension<"I"> {} dim_luminous_intensity;

// base quantities
inline constexpr struct length final : quantity_spec<dim_length> {} length;
inline constexpr struct duration final : quantity_spec<dim_time> {} duration;
inline constexpr struct electric_charge final : quantity_spec<dim_electric_charge> {} electric_charge;
inline constexpr struct energy final : quantity_spec<dim_energy> {} energy;
inline constexpr struct temperature final : quantity_spec<dim_temperature> {} temperature;
inline constexpr struct amount_of_substance final : quantity_spec<dim_amount_of_substance> {} amount_of_substance;
inline constexpr struct luminous_intensity final : quantity_spec<dim_luminous_intensity> {} luminous_intensity;
}
```

Should **`angle`** be a strong dimension or dimensionless?

HEP units

```
inline constexpr struct meter final : named_unit<"m", kind_of<length>> {} meter;
inline constexpr struct second final : named_unit<"s", kind_of<duration>> {} second;
inline constexpr struct hertz final : named_unit<"Hz", one / second, kind_of<frequency>> {} hertz;
inline constexpr struct eplus final : named_unit<symbol_text{u8"e⁺", "e+"}, kind_of<electric_charge>> {} eplus;
inline constexpr struct coulomb final : named_unit<"C", mag<6'241'509'074> * mag_power<10, 9> * eplus> {} coulomb;
inline constexpr struct electronvolt final : named_unit<"eV", kind_of<energy>> {} electronvolt;
inline constexpr struct joule final : named_unit<"J", electronvolt * coulomb / eplus> {} joule;
inline constexpr struct gram final : named_unit<"g", mag_ratio<1, 1000> * joule * square(second) / square(meter)> {} gram;

namespace unit_symbols {

inline constexpr auto m = meter;
inline constexpr auto mm = si::milli<meter>;
inline constexpr auto mm2 = square(mm);
inline constexpr auto mm3 = cubic(mm);

inline constexpr auto ns = si::nano<second>;
inline constexpr auto s = second;
inline constexpr auto ms = si::milli<second>;
inline constexpr auto us = si::micro<second>;

inline constexpr auto eV = electronvolt;
inline constexpr auto MeV = si::mega<electronvolt>;
inline constexpr auto GeV = si::giga<electronvolt>;

}
```

HEP System: Derived Quantities

From energy base to physics quantities

```
using namespace mp_units::hep::unit_symbols;

// Mass derived from energy: M = E/c²
QuantityOf<hep::mass> auto m_e = 0.511 * MeV / c2;  // Electron mass

// Momentum: p = E/c
QuantityOf<hep::momentum> auto p = 45.0 * GeV / c;  // Track momentum
```

HEP System: Derived Quantities

From energy base to physics quantities

```
using namespace mp_units::hep::unit_symbols;

// Mass derived from energy: M = E/c²
QuantityOf<hep::mass> auto m_e = 0.511 * MeV / c2;  // Electron mass

// Momentum: p = E/c
QuantityOf<hep::momentum> auto p = 45.0 * GeV / c;  // Track momentum
```

WHY THIS MATTERS FOR CERN?

- Matches *how physicists think* about particles
- *No confusion* about factors of **c**
- Natural for *relativistic calculations*
- Still allows *conversion to SI mass units when needed*

HEP System: Quantity Subkinds

*Why **`proper_time`** and **`coordinate_time`** must be different kinds?*

```
inline constexpr struct proper_time final : quantity_spec<duration, is_kind> {} proper_time;
inline constexpr struct coordinate_time final : quantity_spec<duration, is_kind> {} coordinate_time;

quantity<hep::lorentz_factor[one]> calculate_lorentz_factor(quantity<hep::coordinate_time[ns]> t_lab,
                                                            quantity<hep::proper_time[ns]> tau)
{ return hep::lorentz_factor(t_lab / tau); }
```

HEP System: Quantity Subkinds

*Why **`proper_time`** and **`coordinate_time`** must be different kinds?*

```
inline constexpr struct proper_time final : quantity_spec<duration, is_kind> {} proper_time;
inline constexpr struct coordinate_time final : quantity_spec<duration, is_kind> {} coordinate_time;

quantity<hep::lorentz_factor[one]> calculate_lorentz_factor(quantity<hep::coordinate_time[ns]> t_lab,
                                                            quantity<hep::proper_time[ns]> tau)
{ return hep::lorentz_factor(t_lab / tau); }
```

```
quantity tau   = hep::proper_time(0.385 * ns);    // B⁰ meson rest-frame lifetime
quantity t_lab = hep::coordinate_time(4.2 * ns);  // observed lab time of flight

// ✅ Division across subkinds — produces dimensionless Lorentz factor
quantity gamma = calculate_lorentz_factor(t_lab, tau);  // γ = t_lab / τ ≈ 10.9
// quantity bad = calculate_lorentz_factor(tau, t_lab); // ❌ Compile error!

// ✅ Explicit cast to plain duration when needed (e.g. for legacy APIs)
quantity t = hep::duration(t_lab);    // explicit — intentional
```

- τ: Lorentz-invariant rest-frame lifetime — **frame-independent**
- t_lab: **frame-dependent lab time** — changes with boost

HEP System: Quantity Subkinds

*Why **`proper_time`** and **`coordinate_time`** must be different kinds?*

```
inline constexpr struct proper_time final : quantity_spec<duration, is_kind> {} proper_time;
inline constexpr struct coordinate_time final : quantity_spec<duration, is_kind> {} coordinate_time;

quantity<hep::lorentz_factor[one]> calculate_lorentz_factor(quantity<hep::coordinate_time[ns]> t_lab,
                                                            quantity<hep::proper_time[ns]> tau)
{ return hep::lorentz_factor(t_lab / tau); }
```

```
quantity tau   = hep::proper_time(0.385 * ns);    // B⁰ meson rest-frame lifetime
quantity t_lab = hep::coordinate_time(4.2 * ns);  // observed lab time of flight

// ✅ Division across subkinds — produces dimensionless Lorentz factor
quantity gamma = calculate_lorentz_factor(t_lab, tau);  // γ = t_lab / τ ≈ 10.9
// quantity bad = calculate_lorentz_factor(tau, t_lab); // ❌ Compile error!

// ✅ Explicit cast to plain duration when needed (e.g. for legacy APIs)
quantity t = hep::duration(t_lab);    // explicit — intentional
```

Subkinds encode physics intent directly into the type system.

HEP System: CODATA Version Control

Ensuring consistent physics constants across frameworks

```
using namespace mp_units::hep::unit_symbols;

// CODATA 2018 (default — matches current Geant4, CLHEP)
std::println("{}", (1. * hep::electron_mass).in(MeV/c2));                    // "0.51099895 MeV/c²"
std::println("{}", (1. * hep::fine_structure_constant).in(one));             // "0.0072973525693"

// CODATA 2022 (matches latest ROOT)
std::println("{}", (1. * hep::codata2022::electron_mass).in(MeV/c2));        // "0.51099895069 MeV/c²"
std::println("{}", (1. * hep::codata2022::fine_structure_constant).in(one)); // "0.0072973525643"

// CODATA 2014 (legacy CLHEP, Gaudi)
std::println("{}", (1. * hep::codata2014::electron_mass).in(MeV/c2));        // "0.5109989461 MeV/c²"
```

HEP System: CODATA Version Control

Ensuring consistent physics constants across frameworks

```
using namespace mp_units::hep::unit_symbols;

// CODATA 2018 (default — matches current Geant4, CLHEP)
std::println("{}", (1. * hep::electron_mass).in(MeV/c2));                    // "0.51099895 MeV/c²"
std::println("{}", (1. * hep::fine_structure_constant).in(one));             // "0.0072973525693"

// CODATA 2022 (matches latest ROOT)
std::println("{}", (1. * hep::codata2022::electron_mass).in(MeV/c2));        // "0.51099895069 MeV/c²"
std::println("{}", (1. * hep::codata2022::fine_structure_constant).in(one)); // "0.0072973525643"

// CODATA 2014 (legacy CLHEP, Gaudi)
std::println("{}", (1. * hep::codata2014::electron_mass).in(MeV/c2));        // "0.5109989461 MeV/c²"
```

Compile-time selection ensures:

- No accidental mixing of constant versions
- Clear documentation of which CODATA is used
- Easy migration path as frameworks update

Potential Migration Strategy for HEP Software

John Chapman (ATLAS Offline Software Coordinator) requirements

- Cannot rewrite 2.5M lines of code overnight
- Must prove zero performance impact
- Inside-out: start with unit headers, expand outward
- Evolutionary, not revolutionary

6-Phase Approach

1. Integrate **mp-units** with your code
2. Drop-in **replacements** for unit constant headers
3. **New code** uses typed quantities from the start
4. Wrap existing interfaces with **type-safe overloads**
5. **Refactor internals**; add typed getters; deprecate **`double`** API
6. **Complete** the migration; remove **`_qty`** suffix

Phase 1: Integrate mp-units with your code

Import mp-units framework and HEP system of quantities and units to your project

```
#include <mp-units/math.h>
#include <mp-units/systems/hep.h>

namespace CLHEP {
  using namespace mp_units;

  namespace mp {
    using namespace mp_units::hep;
    using namespace mp_units::hep::unit_symbols;
  }
}
```

- Pick *any namespace* that suits you for storing strong quantities, units, and constants types
- It should *not collide* with your existing identifiers

Phase 2: Drop-In Replacements for Unit Constant Headers

*Replace hand-maintained **`constexpr double`** constants — all existing code continues to compile*

```
// Before: hand-maintained constants — values must be kept in sync manually
namespace CLHEP {
  static constexpr double mm  = 1.;           // Base unit
  static constexpr double cm  = 10. * mm;     // = 10.0
  static constexpr double MeV = 1.;           // Base unit
  static constexpr double GeV = 1.e+3 * MeV;  // = 1000.0
}
```

Phase 2: Drop-In Replacements for Unit Constant Headers

*Replace hand-maintained **`constexpr double`** constants — all existing code continues to compile*

```
// Before: hand-maintained constants — values must be kept in sync manually
namespace CLHEP {
  static constexpr double mm  = 1.;           // Base unit
  static constexpr double cm  = 10. * mm;     // = 10.0
  static constexpr double MeV = 1.;           // Base unit
  static constexpr double GeV = 1.e+3 * MeV;  // = 1000.0
}
```

```
// After: mp-units derives the values — no manual arithmetic, always correct
namespace CLHEP {
  constexpr double mm  = (1. * mp::mm).numerical_value_in(mp::mm);    // = 1.0  (base unit)
  constexpr double cm  = (1. * mp::cm).numerical_value_in(mp::mm);    // = 10.0
  constexpr double MeV = (1. * mp::MeV).numerical_value_in(mp::MeV);  // = 1.0  (base unit)
  constexpr double GeV = (1. * mp::GeV).numerical_value_in(mp::MeV);  // = 1000.0
}
```

Impact: Existing **`double energy = 50.0 * GeV;`** compiles unchanged

Phase 3: New Code Uses Typed Quantities from the Start

*All newly written functions and classes use **`quantity<...>`** types — legacy code untouched*

```
// New typed function — fully safe, no legacy code touched
quantity<mp::GeV / mp::c> calculate_momentum(quantity<mp::GeV> energy,
                                             quantity<mp::GeV / mp::c2> mass)
{
  return sqrt(pow<2>(energy) - pow<2>(mass * mp::c2)) / mp::c;
}
```

Phase 3: New Code Uses Typed Quantities from the Start

*All newly written functions and classes use **`quantity<...>`** types — legacy code untouched*

```
// New typed function — fully safe, no legacy code touched
quantity<mp::GeV / mp::c> calculate_momentum(quantity<mp::GeV> energy,
                                             quantity<mp::GeV / mp::c2> mass)
{
  return sqrt(pow<2>(energy) - pow<2>(mass * mp::c2)) / mp::c;
}
```

```
// Call site — units enforced at compile time
quantity p = calculate_momentum(125. * mp::GeV,                          // ✅ energy
                                4.18 * mp::GeV / mp::c2);                // ✅ mass
// auto p2 = calculate_momentum(125. * mp::GeV, 4.18 * mp::GeV);         // ❌ Compile error: mass expected
// auto p3 = calculate_momentum(4.18 * mp::GeV / mp::c2, 125. * mp::GeV); // ❌ Compile error: arg order wrong
```

Impact: Dimensional safety from day one, no legacy disruption

Phase 4: Wrap Existing Interfaces with Type-Safe Overloads

*Add typed setter overloads — class internals (raw **`double`** members) left completely unchanged*

```
class Particle {
  double m_mass{};    // Still raw double — internals untouched
  double m_width{};
  double m_charge{};
public:
  // Existing double-based interface — unchanged, still works:
  double getMass() const                { return m_mass; }
  double getWidth() const               { return m_width; }
  double getCharge() const              { return m_charge; }

  void setMass(double mass_MeV_per_c2)  { m_mass   = mass_MeV_per_c2; }
  void setWidth(double width_MeV)       { m_width  = width_MeV; }
  void setCharge(double charge_eplus)   { m_charge = charge_eplus; }

  // New typed overloads — convert at the boundary, delegate to existing setters:
  void setMass(quantity<mp::rest_mass[mp::MeV / mp::c2]> mass)    { setMass(mass.numerical_value_in(mp::MeV / mp::c2)); }
  void setWidth(quantity<mp::decay_width[mp::MeV]> width)         { setWidth(width.numerical_value_in(mp::MeV)); }
  void setCharge(quantity<mp::electric_charge[mp::eplus]> charge) { setCharge(charge.numerical_value_in(mp::eplus)); }
};
```

Phase 4: Wrap Existing Interfaces with Type-Safe Overloads

*Add typed setter overloads — class internals (raw **`double`** members) left completely unchanged*

```
class Particle {
  double m_mass{};    // Still raw double — internals untouched
  double m_width{};
  double m_charge{};
public:
  // Existing double-based interface — unchanged, still works:
  double getMass() const                { return m_mass; }
  double getWidth() const               { return m_width; }
  double getCharge() const              { return m_charge; }

  void setMass(double mass_MeV_per_c2)  { m_mass   = mass_MeV_per_c2; }
  void setWidth(double width_MeV)       { m_width  = width_MeV; }
  void setCharge(double charge_eplus)   { m_charge = charge_eplus; }

  // New typed overloads — convert at the boundary, delegate to existing setters:
  void setMass(quantity<mp::rest_mass[mp::MeV / mp::c2]> mass)    { setMass(mass.numerical_value_in(mp::MeV / mp::c2)); }
  void setWidth(quantity<mp::decay_width[mp::MeV]> width)         { setWidth(width.numerical_value_in(mp::MeV)); }
  void setCharge(quantity<mp::electric_charge[mp::eplus]> charge) { setCharge(charge.numerical_value_in(mp::eplus)); }
};
```

Note: Typed getter overloads cannot be added yet.

Phase 5: Refactor Internals; Add Typed Getters; Deprecate `double` API

*Store typed quantities as members; add **`_qty`** getters; deprecate raw **`double`** API*

```
class Particle {
  quantity<mp::rest_mass[mp::MeV / mp::c2]>  m_mass{};    // Now typed quantities
  quantity<mp::decay_width[mp::MeV]>         m_width{};
  quantity<mp::electric_charge[mp::eplus]>   m_charge{};
public:
  // New typed interface — getters with _qty suffix and setters:
  quantity<mp::rest_mass[mp::MeV / mp::c2]> getMass_qty() const   { return m_mass; }
  quantity<mp::decay_width[mp::MeV]>        getWidth_qty() const  { return m_width; }
  quantity<mp::electric_charge[mp::eplus]>  getCharge_qty() const { return m_charge; }

  void setMass(quantity<mp::rest_mass[mp::MeV / mp::c2]> mass)    { m_mass   = mass; }
  void setWidth(quantity<mp::decay_width[mp::MeV]> width)         { m_width  = width; }
  void setCharge(quantity<mp::electric_charge[mp::eplus]> charge) { m_charge = charge; }

  // Old double interface — now deprecated:
  [[deprecated("Use getMass_qty() instead")]]   double getMass() const   { return m_mass.numerical_value_in(mp::MeV / mp::c2); }
  [[deprecated("Use getWidth_qty() instead")]]  double getWidth() const  { return m_width.numerical_value_in(mp::MeV); }
  [[deprecated("Use getCharge_qty() instead")]] double getCharge() const { return m_charge.numerical_value_in(mp::eplus); }

  [[deprecated("Pass quantity instead")]] void setMass(double mass)      { setMass(mass * mp::MeV / mp::c2); }
  [[deprecated("Pass quantity instead")]] void setWidth(double width)    { setWidth(width * mp::MeV); }
  [[deprecated("Pass quantity instead")]] void setCharge(double charge)  { setCharge(charge * mp::eplus); }
};
```

Phase 5: Refactor Internals; Add Typed Getters; Deprecate `double` API

*Store typed quantities as members; add **`_qty`** getters; deprecate raw **`double`** API*

```
class Particle {
  quantity<mp::rest_mass[mp::MeV / mp::c2]>  m_mass{};    // Now typed quantities
  quantity<mp::decay_width[mp::MeV]>         m_width{};
  quantity<mp::electric_charge[mp::eplus]>   m_charge{};
public:
  // New typed interface — getters with _qty suffix and setters:
  quantity<mp::rest_mass[mp::MeV / mp::c2]> getMass_qty() const   { return m_mass; }
  quantity<mp::decay_width[mp::MeV]>        getWidth_qty() const  { return m_width; }
  quantity<mp::electric_charge[mp::eplus]>  getCharge_qty() const { return m_charge; }

  void setMass(quantity<mp::rest_mass[mp::MeV / mp::c2]> mass)    { m_mass   = mass; }
  void setWidth(quantity<mp::decay_width[mp::MeV]> width)         { m_width  = width; }
  void setCharge(quantity<mp::electric_charge[mp::eplus]> charge) { m_charge = charge; }

  // Old double interface — now deprecated:
  [[deprecated("Use getMass_qty() instead")]]   double getMass() const   { return m_mass.numerical_value_in(mp::MeV / mp::c2); }
  [[deprecated("Use getWidth_qty() instead")]]  double getWidth() const  { return m_width.numerical_value_in(mp::MeV); }
  [[deprecated("Use getCharge_qty() instead")]] double getCharge() const { return m_charge.numerical_value_in(mp::eplus); }

  [[deprecated("Pass quantity instead")]] void setMass(double mass)      { setMass(mass * mp::MeV / mp::c2); }
  [[deprecated("Pass quantity instead")]] void setWidth(double width)    { setWidth(width * mp::MeV); }
```

Old call sites trigger deprecation warnings; new code uses the typed API.

Phase 6: Complete the Migration; Remove `_qty` Suffix

Once all call sites use the typed API, remove deprecated overloads and rename getters

```
class Particle {
  quantity<mp::rest_mass[mp::MeV / mp::c2]> m_mass{};
  quantity<mp::decay_width[mp::MeV]>        m_width{};
  quantity<mp::electric_charge[mp::eplus]>  m_charge{};
public:
  // Fully typed interface — deprecated methods removed, _qty suffix removed:
  quantity<mp::rest_mass[mp::MeV / mp::c2]> getMass() const        { return m_mass; }
  quantity<mp::decay_width[mp::MeV]>        getWidth() const       { return m_width; }
  quantity<mp::electric_charge[mp::eplus]>  getCharge() const      { return m_charge; }

  void setMass(quantity<mp::rest_mass[mp::MeV / mp::c2]> mass)     { m_mass   = mass; }
  void setWidth(quantity<mp::decay_width[mp::MeV]> width)          { m_width  = width; }
  void setCharge(quantity<mp::electric_charge[mp::eplus]> charge)  { m_charge = charge; }
};
```

Phase 6: Complete the Migration; Remove `_qty` Suffix

Once all call sites use the typed API, remove deprecated overloads and rename getters

```
class Particle {
  quantity<mp::rest_mass[mp::MeV / mp::c2]> m_mass{};
  quantity<mp::decay_width[mp::MeV]>        m_width{};
  quantity<mp::electric_charge[mp::eplus]>  m_charge{};
public:
  // Fully typed interface — deprecated methods removed, _qty suffix removed:
  quantity<mp::rest_mass[mp::MeV / mp::c2]> getMass() const        { return m_mass; }
  quantity<mp::decay_width[mp::MeV]>        getWidth() const       { return m_width; }
  quantity<mp::electric_charge[mp::eplus]>  getCharge() const      { return m_charge; }

  void setMass(quantity<mp::rest_mass[mp::MeV / mp::c2]> mass)     { m_mass   = mass; }
  void setWidth(quantity<mp::decay_width[mp::MeV]> width)          { m_width  = width; }
  void setCharge(quantity<mp::electric_charge[mp::eplus]> charge)  { m_charge = charge; }
};
```

This phase is a simple rename: **`sed -i 's/_qty()/()/g'`**.

Performance: Zero Overhead Promise

NO UNITS LIBRARY

```
double ttg_s(double d_m, double v_mps)
{
  return d_m / v_mps;
}
```

MP-UNITS

```
quantity<s> ttg(quantity<m> d, quantity<m/s> v)
{
  return d / v;
}
```

Performance: Zero Overhead Promise

NO UNITS LIBRARY

```
double ttg_s(double d_m, double v_mps)
{
  return d_m / v_mps;
}
```

```
ttg_s(double, double):
  divsd xmm0, xmm1
  ret
```

MP-UNITS

```
quantity<s> ttg(quantity<m> d, quantity<m/s> v)
{
  return d / v;
}
```

```
ttg(quantity<m>, quantity<m/s>):
  divsd xmm0, xmm1
  ret
```

Performance: Zero Overhead Promise

NO UNITS LIBRARY

```
double ttg_s(double d_m, double v_mps)
{
  return d_m / v_mps;
}
```

```
ttg_s(double, double):
  divsd xmm0, xmm1
  ret
```

MP-UNITS

```
quantity<s> ttg(quantity<m> d, quantity<m/s> v)
{
  return d / v;
}
```

```
ttg(quantity<m>, quantity<m/s>):
  divsd xmm0, xmm1
  ret
```

Identical assembly! Units are a compile-time-only concept.

Performance: HEP Unit Conversions

NO UNITS LIBRARY

```
auto foo(double val)
{
  double length = val * CLHEP::mm;
  return length / CLHEP::cm;
}
```

MP-UNITS

```
auto foo(double val)
{
  quantity length = val * mm;
  return length.numerical_value_in(cm);
}
```

Performance: HEP Unit Conversions

NO UNITS LIBRARY

```
auto foo(double val)
{
  double length = val * CLHEP::mm;
  return length / CLHEP::cm;
}
```

```
.LCPI0_0:
  .quad   0x4024000000000000
foo(double):
  divsd   xmm0, qword ptr [rip + .LCPI0_0]
  ret
```

MP-UNITS

```
auto foo(double val)
{
  quantity length = val * mm;
  return length.numerical_value_in(cm);
}
```

```
.LCPI0_0:
  .quad   0x4024000000000000
foo(double):
  divsd   xmm0, qword ptr [rip + .LCPI0_0]
  ret
```

Performance: HEP Unit Conversions

NO UNITS LIBRARY

```
auto foo(double val)
{
  double length = val * CLHEP::mm;
  return length / CLHEP::mm;
}
```

MP-UNITS

```
auto foo(double val)
{
  quantity length = val * mm;
  return length.numerical_value_in(mm);
}
```

Performance: HEP Unit Conversions

NO UNITS LIBRARY

```
auto foo(double val)
{
  double length = val * CLHEP::mm;
  return length / CLHEP::mm;
}
```

```
foo(double):
  ret
```

MP-UNITS

```
auto foo(double val)
{
  quantity length = val * mm;
  return length.numerical_value_in(mm);
}
```

```
foo(double):
  ret
```

Performance: HEP Unit Conversions

NO UNITS LIBRARY

```
auto foo(double val)
{
  double length = val * CLHEP::cm;
  return length / CLHEP::cm;
}
```

MP-UNITS

```
auto foo(double val)
{
  quantity length = val * cm;
  return length.numerical_value_in(cm);
}
```

Performance: HEP Unit Conversions

NO UNITS LIBRARY

```
auto foo(double val)
{
  double length = val * CLHEP::cm;
  return length / CLHEP::cm;
}
```

```
.LCPI0_0:
  .quad   0x4024000000000000
foo(double):
  movsd   xmm1, qword ptr [rip + .LCPI0_0]
  mulsd   xmm0, xmm1
  divsd   xmm0, xmm1
  ret
```

MP-UNITS

```
auto foo(double val)
{
  quantity length = val * cm;
  return length.numerical_value_in(cm);
}
```

```
foo(double):
  ret
```

Standardization: Path to C++ Standard

ISO C++ PAPERS

- P1935: A C++ Approach to Physical Units
- P2980: A motivation, scope, and plan for a physical quantities and units library
- P3045: Quantities and units library

Standardization: Path to C++ Standard

ISO C++ PAPERS

- P1935: A C++ Approach to Physical Units
- P2980: A motivation, scope, and plan for a physical quantities and units library
- P3045: Quantities and units library

CURRENT STATUS

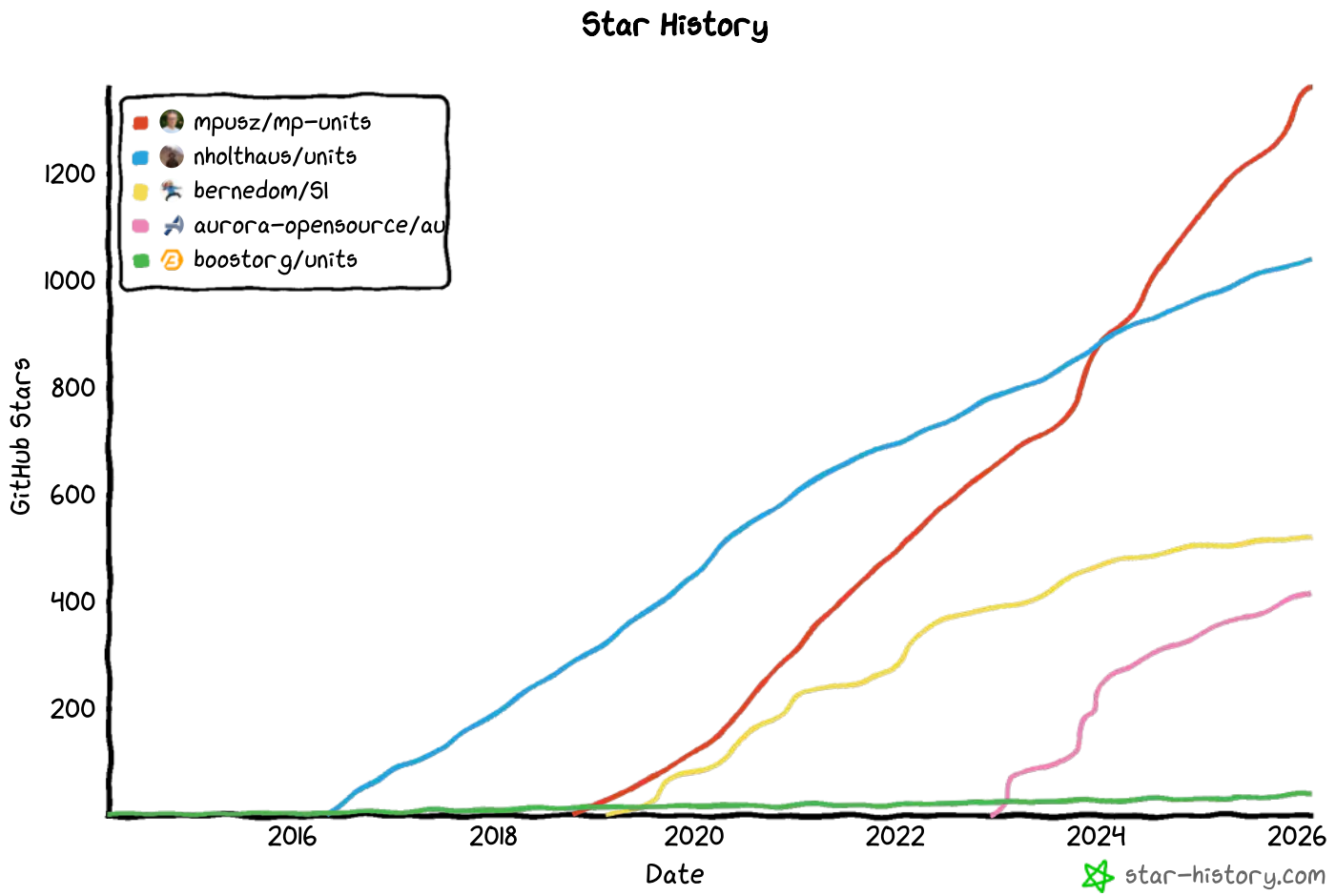
- Presented to LEWG (Library Evolution Working Group)
- Positive feedback from the Committee
- **mp-units** used as reference implementation for discussion
- Target: C++29

Standardization: Path to C++ Standard

WHY THIS MATTERS FOR HEP?

- Investing in future standard library
- Broad C++ community review and feedback
- Eventually part of every compiler
- Skills transferable beyond HEP

Comparison: mp-units vs. Alternatives



Comparison: mp-units vs. Alternatives

SAFETY FEATURE	MP-UNITS (CURRENT)	MP-UNITS (V3 PLANNED)	BOOST.UNITS	NHOLTHAUS	SI	AU
Minimum Requirement	C++20	C++20	C++11/14	C++14	C++17	C++14
Dimension Safety	✅ Full	✅ Full	✅ Full	✅ Full	✅ Full	✅ Full
Unit Safety	✅ Full	✅ Full	⚠️ Partial	⚠️ Partial	⚠️ Limited	✅ Full
Representation Safety	✅ Strong	✅ Strong	⚠️ Partial	⚠️ Partial	⚠️ Limited	⭐ Strong+
Quantity Kind Safety	✅ Full	✅ Full	⚠️ Partial	❌ None	❌ None	❌ None
Quantity Safety	⚠️ Partial	✅ Full	❌ None	❌ None	❌ None	❌ None
Affine Space Safety	✅ Full	✅ Full	⚠️ Partial	❌ None	❌ None	✅ Full
HEP System	✅ Built-in	✅ Built-in	❌ No	❌ No	❌ No	❌ No

Discussion & Next Steps

IMMEDIATE ACTIONS

- **Prototype Phases 1–2**: Namespace setup + drop-in header replacement in Gaudi/CLHEP
- **Benchmark**: Verify zero overhead in ATLAS software stack
- **Pilot Project**: Pick one subsystem (geometry or DAQ) for Phase 3 trial

Discussion & Next Steps

IMMEDIATE ACTIONS

- **Prototype Phases 1–2**: Namespace setup + drop-in header replacement in Gaudi/CLHEP
- **Benchmark**: Verify zero overhead in ATLAS software stack
- **Pilot Project**: Pick one subsystem (geometry or DAQ) for Phase 3 trial

MEDIUM-TERM GOALS

- Develop ROOT/Geant4 bridge with type safety
- Train developers (tutorials, documentation)
- Establish coding guidelines for typed interfaces

Discussion & Next Steps

IMMEDIATE ACTIONS

- **Prototype Phases 1–2**: Namespace setup + drop-in header replacement in Gaudi/CLHEP
- **Benchmark**: Verify zero overhead in ATLAS software stack
- **Pilot Project**: Pick one subsystem (geometry or DAQ) for Phase 3 trial

MEDIUM-TERM GOALS

- Develop ROOT/Geant4 bridge with type safety
- Train developers (tutorials, documentation)
- Establish coding guidelines for typed interfaces

LONG-TERM VISION

- Full typed API across ATLAS/CMS software
- Contribution back to mp-units for HEP community
- Part of C++ standard (C++29)

Key Takeaways

1. **Current CERN code** collapses 7+ physical dimensions into 0-dimensional **`double`**
2. **mp-units** restores those dimensions at **compile-time** with **zero runtime cost**
3. **HEP system** uses energy as base quantity, matching how physicists think
4. **6 levels of safety** catch bugs that training alone cannot prevent
5. **Migration strategy** proven: inside-out, evolutionary not revolutionary
6. **Real-world bugs** (G4Trap, ROOT/Geant4 bridges, CODATA drift) are **preventable**

Key Takeaways

1. **Current CERN code** collapses 7+ physical dimensions into 0-dimensional **`double`**
2. **mp-units** restores those dimensions at **compile-time** with **zero runtime cost**
3. **HEP system** uses energy as base quantity, matching how physicists think
4. **6 levels of safety** catch bugs that training alone cannot prevent
5. **Migration strategy** proven: inside-out, evolutionary not revolutionary
6. **Real-world bugs** (G4Trap, ROOT/Geant4 bridges, CODATA drift) are **preventable**

The compiler can protect scientific integrity—if we give it the information!


CAUTION
Programming
is addictive
(and too much fun)